

Theoretical Framework: **dqi-pricing-benchmark**

Pricing-derived optimization artifacts, exact-reference and exploratory encoding paths, paper-grounded Decoded Quantum Interferometry, decoder/resource diagnostics, and repository-level theory-to-file mapping

Overview

This document states the mathematical framework for the repository in its current structure. It is written to function as a project companion rather than as a detached survey: every major derivation is tied to a concrete notebook, source module, or validation task. The two source-of-truth references are

1. Stephen P. Jordan et al., *Optimization by Decoded Quantum Interferometry*, arXiv:2408.08292.
2. Francesc Sabater et al., *Towards solving industrial integer linear programs with Decoded Quantum Interferometry*, arXiv:2509.08328.

The repository should be read as a layered benchmark with four logically distinct objects.

First, there is a pricing-model layer, in which a discrete vehicle-package pricing problem is specified in a form small enough for exhaustive verification, yet still recognizably motivated by the option-package pricing formulation and demand/constraint language of [2].

Second, there is an encoding layer, which maps the pricing model into a DQI-compatible max-XORSAT artifact. Two paths coexist by design:

- an exact-reference / ILP-derived path, motivated by the 0–1 ILP to max-XORSAT construction of [2], and
- an exploratory Walsh–Hadamard path, which builds a sparse parity surrogate from the exact truth table of the pricing objective.

Third, there is a DQI layer, whose five-step state construction, decoding logic, exact-decoding regime, and finite-size weight theory come from [1] and are interpreted in the industrial decoding/postselection language of [2].

Fourth, there is a benchmark-analysis layer, which quantifies faithfulness, decoder sensitivity, postselection behavior, structure dependence, and resource cost. This layer is not an afterthought: it is the mechanism by which the repository distinguishes paper-faithful statements from exploratory ones.

Two claim boundaries are therefore fundamental.

The first is the distinction between paper-faithful theory and project-specific translation machinery. The max-XORSAT DQI core is inherited from [1], and the ILP-to-max-XORSAT bridge is inherited in spirit from [2]. By contrast, the specific frozen pricing family, the 2-bit tier encoding, the synthetic structure family, and the Walsh-truncation route are repository constructions.

The second is the distinction between mainline and appendix-only deliverables. The exact-reference path is the mainline reviewer-facing backend. The Walsh path is retained as a microscope on approximation quality. The coherent and subset-Qiskit layers remain scoped support tools rather than the headline scientific claim.

This document omits exhaustive discussion of test files unless a test embodies a theoretically nontrivial invariant. The emphasis is instead on `src/` and `notebooks/`.

1. Source-paper bridge

1.1 What is adopted from [1]

Reference [1] introduces DQI as a Fourier-space optimization procedure that prepares a state of the form

$$|P(f)\rangle = \sum_{x \in \{0,1\}^n} P(f(x)) |x\rangle,$$

where f is an objective function over bit strings and P is a degree- ℓ polynomial chosen to bias amplitudes toward high-value assignments. For max-XORSAT, [1] specializes to

$$f(x) = \sum_{i=1}^m (-1)^{v_i + b_i \cdot x},$$

with $B \in GF(2)^{m \times n}$, $v \in GF(2)^m$, and b_i the i th row of B . Equation (2) is the basic objective object used throughout this repository.

The main theoretical ingredients imported from [1] are the following.

1. The DQI five-step state-preparation logic: Dicke-state superposition, phase imprinting by v , syndrome computation $y \rightarrow B^T y$, decoder-based uncomputation, and final Hadamards.

2. The interpretation of the fourth step as a bounded-distance syndrome-decoding problem on the dual code

$$C^\perp = \{d \in GF(2)^m : B^T d = 0\}.$$

3. The exact-decoding uniqueness condition, stated in distance language as

$$2\ell + 1 < d^\perp,$$

where $d^\perp$ is the minimum distance of $C^\perp$.

4. The finite-size weight-selection theory based on the symmetric tridiagonal matrix $A^{(m,\ell,d)}$, whose binary max-XORSAT specialization is $A^{(m,\ell,0)}$.
5. The precise warning that once $2\ell + 1 \geq d^\perp$, the principal eigenvector of that matrix is no longer guaranteed to be the exact finite-instance optimum, though it remains a motivated choice.

1.2 What is adopted from [2]

Reference [2] contributes a different layer. It does not replace the DQI theory of [1]; instead it shows how DQI can be organized around an industrially motivated option-package pricing ILP, transformed to max-XORSAT, and studied with explicit decoder and resource analysis.

The repository adopts from [2] the following methodological structure.

1. The pricing problem should be represented first as a 0–1 optimization model, not as a parity system from the outset.
2. The translation from business optimization to DQI should proceed through an ILP-to-max-XORSAT bridge.
3. Decoder behavior is a first-class object: imperfect decoding changes the pre-postselection state and must be reported, not hidden.
4. Resource accounting should be expressed in terms of problem size, register size, parity structure, decoder settings, and circuit cost.
5. Performance should be interpreted against an exact classical reference, not only against random sampling.

1.3 Repository interpretation of the two papers

This repository does not claim to reproduce the full optimization model of [2] verbatim. Instead, it builds a down-scaled benchmark family that preserves the following logical structure:

- a pricing objective with demand-sensitive revenue and business constraints;
- an ILP-style modeling layer and exact classical solve semantics;
- a translation into max-XORSAT artifacts;
- a DQI execution path grounded in [1];
- decoder, faithfulness, and resource analyses motivated by [2].

This project-specific specialization is legitimate only if the distinction between inherited theory and benchmark-specific simplification remains explicit. That distinction is enforced throughout the present document.

Implementation consequence. The principal source-paper bridge is implemented across `src/pricing_ilp.py`, `src/ilp_to_xorsat_exact.py`, `src/dqi_state.py`, `src/weights_paper.py`, `src/resources.py`, and the notebooks `04_pricing_pipeline.ipynb`, `05_decoder_comparison.ipynb`, `06_resource_analysis.ipynb`, and `08a_paired_encoding_comparison.ipynb` / `08_paired_encoding_comparison.ipynb` depending on the naming convention used in the working tree.

2. Pricing model: benchmark specialization of the pricing problem

2.1 Industrial motivation and benchmark reduction

Reference [2] studies option-package pricing under contribution-margin maximization, discounted bundles, demand migration under price elasticity, and a structured family of package constraints. In that setting, the optimization variables are binary package/option decisions and the natural mathematical object is a 0–1 ILP.

The repository reduces that setting to a compact benchmark family that remains fully enumerable on frozen small instances. The reduction is deliberate. It allows the complete truth table of the benchmark objective to be known exactly, which in turn allows the encoding layer and DQI layer to be audited rather than guessed.

2.2 Discrete tier encoding used in the benchmark

Let there be N_f configurable features. Feature i takes one of three valid tiers:

- standard,
- premium,
- luxury.

The benchmark represents each feature by two bits,

$$00 \rightarrow \text{standard}, 01 \rightarrow \text{premium}, 10 \rightarrow \text{luxury}, 11 \rightarrow \text{invalid}.$$

Hence a full configuration is a bit string

$$x \in \{0, 1\}^n, \quad n = 2N_f.$$

This 2-bit ternary-with-invalid encoding is a benchmark convenience, not an equation from [2]. Its purpose is to preserve a compact combinatorial structure while making invalid states explicit and penalizable.

2.3 Benchmark objective

For the two bits (a_i, b_i) of feature i , define the indicators

$$1_{S,i} = (1 - a_i)(1 - b_i), 1_{P,i} = (1 - a_i)b_i, 1_{L,i} = a_i(1 - b_i), 1_{I,i} = a_i b_i.$$

Let tier prices be $p_i^{(S)}, p_i^{(P)}, p_i^{(L)}$. Then the benchmark package price is

$$\text{Price}(x) = \sum_{i=1}^{N_f} (p_i^{(S)} 1_{S,i} + p_i^{(P)} 1_{P,i} + p_i^{(L)} 1_{L,i}).$$

To mimic the demand response of [2] without reproducing its full package migration model, the benchmark uses a small segmented demand rule. Let segment weights be w_k and price caps be B_k , with $\sum_k w_k = 1$. Define the purchase indicator

$$A_k(x) = 1[\text{Price}(x) \leq B_k].$$

Expected revenue is then

$$R(x) = \sum_k w_k \text{Price}(x) A_k(x).$$

The true benchmark objective is

$$F(x) = R(x) - P_{\text{inv}}(x) - P_{\text{bundle}}(x) - P_{\text{ux}}(x),$$

where the penalty terms enforce invalid-code suppression, bundle/dependency rules, and a luxury-count cap.

This is the first major point where the repository diverges from [2]: the exact industrial pricing equations are replaced by a small fully enumerable analogue. The rationale is methodological rather than cosmetic. Every downstream claim about encoding faithfulness and DQI performance is stronger when the originating objective is exactly known.

2.4 ILP interpretation

The benchmark objective (11) is still meant to be viewed through the lens of the 0–1 formulation emphasized in [2]. Abstractly, the optimization layer takes the form

$$\max_{x \in \{0,1\}^n} c^T x$$

subject to

$$Ax \leq b, \quad A_{=}x = b_{=}.$$

possibly after the introduction of auxiliary binary variables that linearize benchmark-specific logic. Equation (12) is the correct modeling contract for the repository’s pricing layer. It is not necessary that every notebook expose the full auxiliary-variable expansion; what is necessary is that the semantics of the exact solver and the semantics of the exported artifact agree.

Implementation consequence. Theoretical responsibility for this section lies with `src/problem_generator.py`, `src/pricing_ilp.py`, `src/pricing_model_or_tools.py`, and, for the dynamic extension, `src/problem_generator_multiperiod.py` and `src/pricing_model_or_tools_multiperiod.py`. The corresponding notebooks are `02_pricing_instance_demo.ipynb`, `04_pricing_pipeline.ipynb`, and `09_3period_pricing_extension.ipynb`.

3. Exact classical semantics and frozen instance families

3.1 Why the exact baseline is part of the theory

A central lesson of [2] is that DQI performance should be interpreted against an exact classical reference. At benchmark scale, this repository strengthens that idea further: exact classical semantics are not merely a performance baseline but a semantic anchor. They determine what the correct objective value is, what the correct optimum is, and whether an encoding preserves the original optimization task.

3.2 Frozen families

The repository is organized around several frozen instance families.

- Family P: one-period pricing instances, used to study pricing-to-artifact translation and DQI behavior on pricing-derived rows.
- Family C: tiny exact or coherent validation artifacts, used to validate statevector-level or exact-reference constructions where joint-state reasoning is tractable.
- Family S: synthetic structured max-XORSAT instances, used to probe how decoder performance depends on parity-check structure rather than on pricing semantics.

The role of these families is theoretically clean.

Family P tests whether the full optimization-to-DQI chain is faithful.

Family C tests whether tiny-state constructions match the expected quantum object.

Family S isolates structure variables that are conflated inside pricing-derived instances.

3.3 Exact solver contract

The repository uses brute force on tiny frozen instances and CP-SAT on pricing instances as the semantic source of truth for the optimization layer. This is the correct interpretation of `src/classical_baselines.py` and `src/pricing_model_or_tools.py`: they are not merely convenience baselines but the repository’s exact reference for what the pricing model means.

Implementation consequence. The exact baseline layer is implemented in `src/classical_baselines.py`, `src/classical_baselines_multiperiod.py`, `src/pricing_model_or_tools.py`, and `src/pricing_model_or_tools_multiperiod.py`, with benchmark orchestration in `src/benchmark.py` and `src/matrix_runner.py`.

4. Two encoding paths from pricing semantics to max-XORSAT artifacts

This repository contains two mathematically distinct translations from the pricing model to a DQI-ready artifact. They must not be narrated as equivalent.

4.1 Exact-reference / ILP-derived path

Reference [2] formalizes the general 0–1 ILP problem and explains that mapping such an instance to max-XORSAT can be decomposed into two conceptual stages:

1. convert feasibility constraints to an equivalent parity-satisfaction instance, and
2. reduce objective maximization to a sequence of max-XORSAT or SAT-like feasibility calls.

This motivates the repository’s exact-reference path, implemented in `src/ilp_to_xorsat_exact.py`. The theoretical goal of this path is not approximation but preservation: if the pricing problem has a definite optimum, the translated artifact should preserve that optimum and the relevant ranking structure, modulo explicitly documented auxiliary-variable conventions.

This is the correct reviewer-facing path because it aligns with the architecture advocated in [2]:

pricing model → exact optimization semantics → max-XORSAT instance → DQI.

4.2 Exploratory Walsh–Hadamard path

The Walsh path begins from the exact truth table of the benchmark objective F rather than from the ILP contract. Expand

$$F(x) = \sum_{S \subseteq [n]} \hat{F}(S) (-1)^{\langle S, x \rangle},$$

with Walsh coefficients

$$\hat{F}(S) = 2^{-n} \sum_{x \in \{0,1\}^n} F(x) (-1)^{\langle S, x \rangle}.$$

Retain the top- K nonconstant coefficients by magnitude, producing the weighted surrogate

$$G_{wt}(x) = \sum_{S \in \mathcal{S}_K} \hat{F}(S)(-1)^{\langle S, x \rangle}.$$

The native DQI max-XORSAT input is unweighted, so a further sign-only simplification gives

$$G_{unw}(x) = \sum_{i=1}^m (-1)^{v_i + b_i \cdot x}.$$

This route is useful because it creates a transparent microscope on spectral sparsity, retained energy, and transfer failure. It is not paper-faithful in the sense of [2], because the parity instance is being inferred from the truth table rather than from the optimization model.

4.3 Consequence for repository semantics

The existence of the two paths is not redundant. It enables two different scientific questions.

- The exact-reference path asks: can the optimization model be translated into a DQI-compatible instance without semantic loss?
- The Walsh path asks: how much of the true pricing objective is captured by a sparse parity surrogate, and where does transfer fail?

This distinction motivates the dedicated comparison layer in `src/encoding_compare.py`, `src/verify_encoding.py`, `src/faithfulness.py`, and `src/metrics_decision.py`.

Implementation consequence. `src/ilp_to_xorsat_exact.py` is the mainline encoding module. `src/reduction.py` is explicitly exploratory. `src/encoding_compare.py` is the module that turns this distinction into a measurable benchmark claim. The notebooks `04_pricing_pipeline.ipynb` and `08a_paired_encoding_comparison.ipynb` / `08_paired_encoding_comparison.ipynb` are the principal user-facing demonstrations of the difference.

5. Max-XORSAT objective and DQI target state

5.1 Max-XORSAT as the native DQI object

Once an encoding path outputs (B, v) with $B \in GF(2)^{m \times n}$ and $v \in GF(2)^m$, the repository enters the theory of [1]. The native objective is the max-XORSAT score

$$f(x) = \sum_{i=1}^m (-1)^{v_i + b_i \cdot x}.$$

If $s(x)$ is the number of satisfied parity checks, then

$$f(x) = 2s(x) - m, \quad s(x) = \frac{f(x) + m}{2}.$$

Thus maximizing f is exactly equivalent to maximizing the number of satisfied constraints.

5.2 Polynomial amplification target

DQI does not merely sample from $f(x)^2$. It prepares a state with amplitudes given by a degree- ℓ polynomial in the objective value,

$$|P(f)\rangle = \sum_{x \in \{0,1\}^n} P(f(x)) |x\rangle.$$

Measurement of this state in the computational basis yields probability

$$\Pr(x) = \frac{|P(f(x))|^2}{\sum_z |P(f(z))|^2}.$$

The purpose of the degree cutoff ℓ is therefore precise: it controls how strongly amplitudes concentrate on assignments with large objective value, while simultaneously controlling the maximum error weight that the decoder must reliably handle.

5.3 Elementary-symmetric decomposition

The state (19) can be expressed in the form used by [1], namely as a superposition over weight classes of an error string $y \in \{0, 1\}^m$,

$$\sum_{k=0}^{\ell} \frac{w_k}{\sqrt{\binom{m}{k}}} \sum_{|y|=k} (-1)^{v \cdot y} |B^T y\rangle,$$

before the final Hadamards are applied. This representation is what justifies the Dicke-state preparation strategy and the decoding interpretation of the fourth step.

Implementation consequence. The mathematical object of this section is realized in `src/dqi_state.py`, while the surrounding workflow of sampling and evaluation is handled in `src/dqi_pipeline.py`.

6. The five-step DQI construction

Reference [1] describes the DQI preparation of $|P(f)\rangle$ in five steps. This section is the theoretical basis for `src/dqi_state.py`.

6.1 Step 1: Dicke-state superposition

For $0 \leq k \leq \ell$, define the Hamming-weight- k Dicke state on m qubits by

$$|D_{m,k}\rangle = \frac{1}{\sqrt{\binom{m}{k}}} \sum_{y \in \{0,1\}^m: |y|=k} |y\rangle.$$

The first step prepares

$$|\Psi_1\rangle = \sum_{k=0}^{\ell} w_k |D_{m,k}\rangle.$$

The coefficients w_k are the degree-allocation weights discussed later in Section 8.

6.2 Step 2: phase imprinting by the clause signs

Applying Z_i whenever $v_i = 1$ gives

$$|\Psi_2\rangle = \sum_{k=0}^{\ell} \frac{w_k}{\sqrt{\binom{m}{k}}} \sum_{|y|=k} (-1)^{v \cdot y} |y\rangle.$$

This is the phase-oracle role of v .

6.3 Step 3: syndrome computation

A reversible matrix-multiplication subroutine computes the syndrome

$$s = B^T y \in GF(2)^n$$

into an ancilla register, producing

$$|\Psi_3\rangle = \sum_{k=0}^{\ell} \frac{w_k}{\sqrt{\binom{m}{k}}} \sum_{|y|=k} (-1)^{v \cdot y} |y\rangle |B^T y\rangle.$$

6.4 Step 4: decoder-based uncomputation

The fourth step attempts to infer y from its syndrome $B^T y$, using the promise $|y| \leq \ell$. This is the bounded-distance syndrome-decoding problem on the dual code (3). In the ideal regime, the decoder recovers y and uncomputes the message register to zero:

$$|\Psi_4\rangle = \sum_{k=0}^{\ell} \frac{w_k}{\sqrt{\binom{m}{k}}} \sum_{|y|=k} (-1)^{v \cdot y} |0\rangle^{\otimes m} |B^T y\rangle.$$

6.5 Step 5: final Hadamards

Applying $H^{\otimes n}$ to the syndrome register yields the target state (19), up to normalization.

6.6 Exact and approximate realizations

The preceding derivation is exact only when uncomputation succeeds on the entire support of the prepared superposition. This is the reason the decoder is not merely an implementation detail. It is mathematically part of the state-preparation procedure.

Implementation consequence. `src/dqi_state.py` is responsible for the mathematical content of this section. `src/dicke_circuit.py` exists because Step 1 is nontrivial and must be given either an explicit circuit construction or a clearly labeled analytic resource model. `01_reproduce_toy_dqi.ipynb` exists to make these five steps transparent on a tiny instance before pricing-derived complications are introduced.

7. Exact-decoding regime, imperfect decoding, and postselection

7.1 Exact-decoding regime

Let $d^\perp$ be the minimum distance of the dual code $C^\perp$. If

$$2\ell + 1 < d^\perp,$$

then any error pattern of weight at most ℓ has a unique syndrome, and the decoder-based uncomputation logic is exact in principle.

At the repository's small scale, it is useful to strengthen this criterion operationally. Let

$$E_\ell = \{y \in \{0,1\}^m : |y| \leq \ell\}.$$

The implementation can explicitly check whether the map

$$y \rightarrow B^T y$$

is injective on E_ℓ . When this holds, the repository may label the run as exact mode. Otherwise it is approximate mode.

7.2 Imperfect decoding in the language of [2]

Reference [2] makes explicit what imperfect decoding does to the quantum state. If D_k is the set of weight- k error strings decoded successfully and F_k the set decoded unsuccessfully, then before postselection one has a state of the form

$$\sum_{k=0}^{\ell} \omega_k \frac{1}{\sqrt{\binom{m}{k}}} \left(\sum_{y \in D_k} (-1)^{v \cdot y} |0\rangle^{\otimes m} |B^T y\rangle + \sum_{y \in F_k} (-1)^{v \cdot y} |y \oplus y'\rangle |B^T y\rangle \right).$$

The DQI protocol then measures the message register and keeps only the all-zero branch. The renormalized postselected state is therefore

$$\frac{1}{R} \sum_{k=0}^{\ell} \omega_k \frac{1}{\sqrt{\binom{m}{k}}} \sum_{y \in D_k} (-1)^{v \cdot y} |B^T y\rangle,$$

where R is the postselection normalization factor.

7.3 Meaning of postselection success

The postselection probability is therefore not an incidental implementation statistic. It is the total surviving weight of the branch that correctly realizes the intended DQI interference pattern. Reporting it is theoretically mandatory whenever the decoder is imperfect.

7.4 Decoder hierarchy in the repository

The repository's decoder modules embody different approximations to the same mathematical task.

- `src/decoder_bruteforce.py` implements bounded-distance enumeration and serves as the exact small-instance reference.
- `src/decoder_oracle.py` is a tiny-instance exact or near-exact decoder utility.
- `src/decoder_bp1.py` implements hard-decision belief propagation, motivated directly by the decoder discussion in [2].
- `src/decoder_bp_osd.py` extends the heuristic decoder family with a stronger ordered-statistics-assisted variant.
- `src/decoder_study_runner.py` exists to make decoder choice a controlled scientific variable rather than a hidden implementation toggle.

7.5 Why the structure family is theoretically justified

Because syndrome decoding depends strongly on parity-check structure, the repository includes Family S and `src/family_s_generator.py`. This is not a side project. It isolates variables such as density, collision rate, and rank profile that influence decoder success independently of pricing semantics.

Implementation consequence. The notebooks `05_decoder_comparison.ipynb` and `09_decoder_phase_diagram.ipynb` are the natural theory demonstrations for this section. They answer two different questions: decoder sensitivity on pricing-derived artifacts and decoder sensitivity across structure-controlled synthetic artifacts.

8. Finite-size weight theory and the alpha modules

8.1 Paper-derived matrix

The degree-allocation vector

$$w = (w_0, w_1, \dots, w_\ell)^T, \quad \|w\|_2 = 1,$$

determines the Dicke-state superposition. In [1], the finite-size max-XORSAT theory is expressed using the symmetric tridiagonal matrix

The matrix $A^{(m,\ell,d)}$ is tridiagonal with main diagonal $(0, d, 2d, \dots, \ell d)$ and first off-diagonals $(a_1, a_2, \dots, a_\ell)$.

with

$$a_k = \sqrt{k(m-k+1)}, \quad k = 1, \dots, \ell.$$

For the binary balanced max-XORSAT setting relevant here, $d = 0$, so the operative matrix is

The matrix $A^{(m,\ell,0)}$ is tridiagonal with main diagonal $(0, 0, \dots, 0)$ and first off-diagonals $(a_1, a_2, \dots, a_\ell)$.

8.2 Exact-regime optimization rule

In the exact-decoding regime, [1] shows that the expected number of satisfied constraints takes the form

$$\langle s \rangle = \frac{m}{2} + \frac{1}{2} w^\dagger A^{(m,\ell,0)} w.$$

Hence the paper-faithful finite-size choice of w is the solution of the Rayleigh-quotient problem

$$w^\star = \text{principal_eigenvector}(A^{(m,\ell,0)}).$$

This is the theoretical foundation of `src/weights_paper.py`.

8.3 Exact-mode versus approximate-mode interpretation

Reference [1] is explicit that when

$$2\ell + 1 \geq d^\perp,$$

the principal eigenvector of $A^{(m,\ell,0)}$ is not guaranteed to remain the exact finite-instance optimum. The repository must therefore distinguish two uses of paper-derived alpha weights:

- in exact mode, they are paper-faithful finite-size optima;
- in approximate mode, they are paper-motivated heuristics.

8.4 Uniform and heuristic alternatives

The repository contains additional weight policies for ablation.

The baseline uniform choice is

$$w_k^{unif} = \frac{1}{\sqrt{\ell+1}}, \quad 0 \leq k \leq \ell.$$

Any non-paper matrix or proxy rule should be confined to `src/weights_heuristic.py` and clearly labeled as such. `src/weights.py` is then best interpreted as a compatibility shim exposing the weight policies without obscuring their origin.

8.5 Theoretical role of alpha-validation notebooks

`10_alpha_finite_size_validation.ipynb` and `src/alpha_theory_validation.py` exist because the weight-selection rule is not merely a hyperparameter choice. It is a direct mathematical consequence of the finite-size theory of [1] and therefore deserves its own validation layer.

Implementation consequence. The alpha layer consists of `src/weights.py`, `src/weights_paper.py`, `src/weights_heuristic.py`, and `src/alpha_theory_validation.py`, with notebook support in `03_paper_faithful_dqi.ipynb` and `10_alpha_finite_size_validation.ipynb`.

9. Resource model and the role of `resources.py`

9.1 Why resources are part of the theoretical framework

Reference [2] treats logical quantum resource estimation as a central methodological layer. This repository follows that principle on small verified instances. Resource analysis belongs in the theory document because the dominant costs are visible directly in the mathematics:

- the message register has size m ;
- the syndrome register has size n ;
- syndrome computation cost depends on the number of nonzero entries of B ;
- decoder cost depends on the decoder family, error radius ℓ , and iteration count;
- Dicke-state preparation contributes nontrivial overhead before decoding even begins.

9.2 Register accounting

For a max-XORSAT instance (B, v) with $B \in GF(2)^{m \times n}$, the conceptual minimum is

- m message qubits,
- n syndrome qubits,
- ancillas for state preparation and decoding.

If a realization keeps explicit iteration, comparator, or auxiliary registers for decoding, those must be counted separately.

9.3 Syndrome cost

A direct reversible implementation of $y \rightarrow B^T y$ by CNOT fan-out incurs a leading CNOT count equal to the number of ones in B ,

$$N_{CNOT}^{syndrome} = \|B\|_0.$$

This is one of the simplest and most interpretable structure-to-resource bridges in the repository.

9.4 Dicke-state cost

Reference [1] states that the required Dicke-state superpositions can be prepared using $O(m^2)$ gates. The repository's `src/dicke_circuit.py` should therefore be interpreted as either

1. an explicit small- m construction realizing this step, or
2. an analytic cost model for the same step when a full construction is not required for the benchmark layer.

9.5 Decode-inclusive versus subset-only numbers

A vital reporting distinction must be preserved.

- Analytic resource estimates in `src/resources.py` may be decode-inclusive.
- Qiskit transpiled numbers from `src/dqi_circuit_qiskit.py` and `07_qiskit_structural_parity.ipynb` may cover only the implemented structural subset.

These are not interchangeable. The framework should never let a subset-only transpiled count masquerade as a full-stack DQI cost.

9.6 Quality-vs-cost comparison

The existence of `src/resources_compare.py` and `12_quality_vs_cost.ipynb` is theoretically justified because resource numbers become meaningful only when joined to quality metrics such as best recovered F , postselection success, or decision regret. Resource tables without quality context, and quality tables without cost context, both leave the central trade-off unspecified.

Implementation consequence. This section maps to `src/resources.py`, `src/dicke_circuit.py`, `src/resources_compare.py`, `notebooks/06_resource_analysis.ipynb`, and `notebooks/12_quality_vs_cost.ipynb`.

10. Structural Qiskit validation

10.1 Correct scope

The Qiskit layer is best understood as a structural cross-framework validation, not as a replacement implementation of the entire repository. The mathematical question it addresses is narrow and precise: does the implemented circuit subset realize the same statevector-level transformation as the reference state-preparation logic?

10.2 Appropriate parity metric

For any shared circuit subset, a natural agreement measure is total variation distance between output distributions,

$$d_{TV}(p, q) = \frac{1}{2} \sum_x |p(x) - q(x)|,$$

or, more strongly, statevector overlap up to global phase.

10.3 Why the subset limitation matters

The repository’s Qiskit port should be described as validating the implemented subset only. If `decode/uncompute` is absent from that notebook, then the notebook validates the structural circuit pieces it contains, not the full DQI execution stack.

This restriction is not a weakness; it is correct scope control. The role of `src/dqi_circuit_qiskit.py` is to check that the project’s quantum-circuit semantics do not secretly depend on one software stack.

Implementation consequence. This section maps to `src/dqi_circuit_qiskit.py` and `07_qiskit_structural_parity.ipynb`.

11. Coherent tiny-instance validation

11.1 Motivation

The file map contains a coherent-validation branch built around `src/coherent_reference.py` and `08b_coherent_validation.ipynb`. Its theoretical purpose is narrow: to provide a tiny-instance sanity layer for state constructions that become too large to reason about coherently at benchmark scale.

11.2 Claim boundary

This branch should remain appendix-only. It is not the mainline pricing-to-DQI story. Its value lies in checking exact small-instance quantum objects, not in carrying the repository’s principal application claim.

11.3 Why it still belongs in the framework

A theory document that omits the coherent branch would leave `src/coherent_reference.py` and `08b_coherent_validation.ipynb` orphaned. Their correct interpretation is that they verify exactness boundaries on tiny artifacts where the intended state can be compared directly against an independently constructed reference.

Implementation consequence. The coherent layer maps to `src/coherent_reference.py` and `08b_coherent_validation.ipynb`.

12. Dynamic extension: three-period pricing

12.1 Purpose of the extension

The three-period extension in `src/problem_generator_multiperiod.py`, `src/pricing_model_or_tools_multiperiod.py`, and `src/classical_base.py` should not be treated as a separate project. It is a controlled extension of the pricing layer that asks whether the same architecture—optimization model, exact reference, artifact export, and downstream benchmarking—can be extended beyond a one-period static setting.

12.2 Generic form

A generic multi-period analogue of the one-period benchmark objective is

$$F_{mp}(x^{(1)}, \dots, x^{(T)}) = \sum_{t=1}^T R_t(x^{(t)}) - \sum_{t=1}^T P_t(x^{(t)}) - P_{inventory} - P_{smoothing}.$$

This captures period-indexed demand/revenue terms, shared inventory or coupling constraints, and optional smoothness or switching penalties.

12.3 Why the extension is theoretically subordinate

The dynamic extension is subordinate because the source papers are not about this three-period benchmark per se. Its role is to demonstrate that the repository’s modeling contract is extensible, not to dilute the mainline theory built around the one-period pricing benchmark and the DQI artifacts derived from it.

Implementation consequence. This section maps to `src/problem_generator_multiperiod.py`, `src/pricing_model_or_tools_multiperiod.py`, `src/classical_baselines_multiperiod.py`, and `09_3period_pricing_extension.ipynb`.

13. Faithfulness, decision metrics, and benchmark orchestration

13.1 Why faithfulness is a separate theoretical object

The repository distinguishes between optimizing a surrogate and recovering good decisions for the true pricing objective. This distinction is theoretically necessary because the exploratory Walsh path introduces at least two approximation layers: coefficient truncation and unweighting. Even the exact-reference path must still be checked for semantic preservation under export conventions and bit ordering.

13.2 Canonical metric families

The relevant metric families include:

- objective preservation and regret,
- top- k optimum recall,
- rank correlation between F and the surrogate score,
- retained spectral energy for WHT surrogates,
- decision distortion after transferring sampled assignments back to F ,
- postselection success and decode success.

These are the natural responsibilities of `src/metrics_decision.py` and `src/faithfulness.py`.

13.3 Orchestration as a mathematical necessity

`src/benchmark.py`, `src/matrix_runner.py`, `src/report_schema.py`, `src/boundary.py`, and `src/notebook_utils.py` are not merely project plumbing. They exist because a benchmark that compares multiple families, encoding paths, decoders, and weight policies requires consistent experimental semantics. Without a schema boundary, comparison tables become mathematically ambiguous: a row may mix incomparable decoding regimes, resource conventions, or objective definitions.

13.4 Role of the dashboard notebooks

- `11_decision_faithfulness_dashboard.ipynb` aggregates the faithfulness layer.
- `12_quality_vs_cost.ipynb` aggregates the quality/resource trade-off layer.
- `13_scaling_analysis.ipynb` aggregates growth trends in size, cost, or performance proxies across families.

Their purpose is not to introduce new theory, but to make existing theoretical distinctions visible at the benchmark level.

Implementation consequence. The core modules are `src/benchmark.py`, `src/matrix_runner.py`, `src/metrics_decision.py`, `src/faithfulness.py`, `src/report_schema.py`, `src/boundary.py`, and `src/notebook_utils.py`. The principal notebooks are `11_decision_faithfulness_dashboard.ipynb`, `12_quality_vs_cost.ipynb`, and `13_scaling_analysis.ipynb`.

14. Notebook-by-notebook theoretical motivation

This section documents why each notebook exists from the standpoint of the mathematics.

14.1 `01_reproduce_toy_dqi.ipynb`

This notebook is the canonical minimal demonstration of the five-step DQI mechanism from [1]. Its purpose is not application realism but conceptual transparency. It should explain how the Dicke superposition, phase imprinting, syndrome computation, uncomputation, and Hadamard transform combine to bias measurement outcomes toward high-value assignments.

14.2 `02_pricing_instance_demo.ipynb`

This notebook introduces the benchmark pricing family itself. It should make the objective F , the bit encoding, the feasibility rules, and the exact small-instance semantics concrete and human-readable. Without this notebook, the repository would appear to start from artifacts rather than from a modeling problem.

14.3 **03_paper_faithful_dqi.ipynb**

This notebook is the closest direct microscope on the max-XORSAT DQI core. Its role is to isolate the paper-grounded DQI logic on explicit small artifacts, including alpha choices and decoder/postselection behavior, without conflating those with the broader pricing-model pipeline.

14.4 **04_pricing_pipeline.ipynb**

This is the primary bridge notebook from optimization semantics to DQI artifacts. It should begin from the pricing instance, solve it classically, serialize the artifact, and expose both the exact-reference and exploratory encoding handoff. In theoretical terms, it enforces the sequencing pricing model $\rightarrow$ exact baseline $\rightarrow$ exported artifact.

14.5 **05_decoder_comparison.ipynb**

This notebook studies the mathematical dependence of the DQI state-preparation success on the decoder family. Its correct question is not simply “which decoder is better?” but “how does decoder quality alter postselection mass and the best recovered true objective under fixed artifact and fixed ℓ ?”

14.6 **06_resource_analysis.ipynb**

This notebook realizes the resource model of Section 9. It should make explicit which counts are analytic, which are transpiled, and which circuit pieces are included. Its theoretical point is that problem structure, decoder radius, and resource cost are coupled.

14.7 **07_qiskit_structural_parity.ipynb**

This notebook is the structural cross-framework check. It should be read as validating the implemented circuit subset, not as reproducing the full benchmark stack in a second framework.

14.8 **08a_paired_encoding_comparison.ipynb / 08_paired_encoding_comparison.ipynb**

This notebook compares the exact-reference and WHT encoding paths. It exists because the repository makes a methodological distinction between faithful translation and exploratory spectral approximation. This notebook is the explicit mathematical defense of that distinction.

14.9 **08b_coherent_validation.ipynb**

This notebook supports the appendix-only coherent branch discussed in Section 11. Its role is to validate tiny coherent constructions where direct exact comparison is possible.

14.10 **09_decoder_phase_diagram.ipynb**

This notebook isolates structure variables by moving from pricing-derived artifacts to controlled synthetic parity families. The corresponding theory question is how decoder performance varies across regions of structure space, not merely across pricing instances.

14.11 **09_3period_pricing_extension.ipynb**

This notebook demonstrates that the pricing-model contract can be extended to a small multi-period setting without changing the overall benchmark architecture.

14.12 **10_alpha_finite_size_validation.ipynb**

This notebook validates the paper-derived weight logic. Its purpose is to compare uniform, paper-derived, and optional heuristic alpha policies in the light of the exact/approximate regime distinction.

14.13 **11_decision_faithfulness_dashboard.ipynb**

This notebook is the aggregated transparency layer for the decision-quality metrics. It should expose where an artifact is semantically faithful, where approximation enters, and which rows are not comparable.

14.14 **12_quality_vs_cost.ipynb**

This notebook joins quality metrics to resource metrics. It is the place where the practical significance of a resource count becomes visible.

14.15 **13_scaling_analysis.ipynb**

This notebook studies how cost and selected quality proxies vary with size parameters such as n , m , or support size $\sum_{k=0}^{\ell} \binom{m}{k}$. It is the correct place to display growth trends without overclaiming asymptotic performance from small-scale data.

15. Source-module theoretical motivation

15.1 Problem-domain modules

`src/problem_generator.py` defines the one-period benchmark family and therefore the true objective F against which all artifact paths are judged.

`src/pricing_ilp.py` defines the canonical optimization-model contract. The repository should treat this contract as the semantic boundary between domain logic and encoding logic.

`src/pricing_model_or_tools.py` is the exact-solver interpretation of the one-period benchmark. It is the single source of truth for solve semantics when a CP-SAT formulation is used.

`src/problem_generator_multiperiod.py` and `src/pricing_model_or_tools_multiperiod.py` extend the same modeling contract to the dynamic setting of Section 12.

15.2 Encoding modules

`src/ilp_to_xorsat_exact.py` implements the mainline encoding path inspired by the 0–1 ILP to max-XORSAT logic of [2]. It should be regarded as the scientifically preferred backend.

`src/reduction.py` implements the exploratory Walsh-based surrogate construction. Its role is diagnostic, not foundational.

`src/verify_encoding.py` exists to check that the exported artifact still means what the pricing model meant. This is a semantic validation layer, not a formatting layer.

`src/encoding_compare.py` exists because the two encoding paths are not equivalent and should be benchmarked explicitly.

15.3 DQI-core modules

`src/dqi_state.py` implements the mathematical content of Sections 5–7.

`src/dqi_pipeline.py` is the state-preparation-to-sampling workflow: prepare the state, invoke a decoder policy, postselect when required, sample, and score the resulting assignments under the relevant objectives.

`src/dicke_circuit.py` implements or parameterizes the nontrivial Dicke-state preparation subroutine.

`src/dqi_circuit_qiskit.py` re-expresses a structural subset of the same quantum logic in Qiskit for cross-framework validation.

15.4 Decoder modules

`src/decoder_bruteforce.py`, `src/decoder_oracle.py`, `src/decoder_bp1.py`, `src/decoder_bp_osd.py`, and `src/decoder_study_runner.py` together represent the decoder family abstraction. The theory document treats them as alternative realizations of the same bounded-distance inference task, with different faithfulness/cost trade-offs.

15.5 Weight modules

`src/weights_paper.py` is the repository’s direct implementation of the finite-size matrix logic of [1].

`src/weights_heuristic.py` contains intentionally non-paper alternatives and should remain clearly labeled as such.

`src/weights.py` is the shared interface that exposes the alpha policies to the rest of the repository without erasing their theoretical origin.

`src/alpha_theory_validation.py` exists because alpha choice is part of the mathematics, not just part of tuning.

15.6 Analysis and reporting modules

`src/resources.py` converts the circuit logic into qubit, depth, and gate-count estimates.

`src/resources_compare.py` forms a quality-resource join layer.

`src/structure_metrics.py` translates parity-check matrices into interpretable structural diagnostics relevant to decoding.

`src/metrics_decision.py` defines the semantic comparison metrics between pricing objective and artifact behavior.

`src/faithfulness.py` normalizes and aggregates semantic-preservation metrics.

`src/benchmark.py` and `src/matrix_runner.py` orchestrate family-level experiments and encode the rules of comparability.

`src/report_schema.py`, `src/boundary.py`, and `src/notebook_utils.py` keep benchmark semantics stable across notebooks and scripted runs.

15.7 Family and data modules

`src/coherent_reference.py` supports tiny coherent exactness checks.

`src/family_s_generator.py` creates the synthetic structure family required for decoder phase studies.

`src/family_s_manifest.py` validates the artifacts for that family.

16. Validation criteria

The repository’s theory implies the following validation obligations.

16.1 Pricing semantics

For frozen small pricing instances, exact classical baselines must agree on the optimum and bit ordering. This validates the domain layer independently of DQI.

16.2 Encoding faithfulness

For the exact-reference path, one should check objective preservation, regret, optimum recall, and bit-order consistency.

For the Walsh path, one should additionally check retained spectral energy, rank correlation, and the gap between weighted and unweighted surrogates.

16.3 Decoder integrity

Decoder studies must report at least postselection success, decode success by radius, and the best recovered true objective. When approximate mode is entered, that fact must be stated explicitly.

16.4 Alpha correctness

For `weights_paper.py`, one should validate symmetry of the implemented matrix, eigenpair residuals, and the exact/approximate interpretation of the resulting weight vector.

16.5 Qiskit scope control

Subset-only structural validation must not be rewritten as full-stack parity.

16.6 Resource reporting

Every reported resource number should specify whether it is analytic or transpiled, decode-inclusive or subset-only.

Implementation consequence. The notebooks 02, 04, 05, 06, 07, 08a/08, 09, 10, 11, 12, and 13 collectively span these validation obligations.

17. Capstone mapping table

Theory object	Key equations	Primary source files	Primary notebooks	Claim supported
Benchmark pricing objective	(5)–(12)	<code>problem_generator.py</code> , <code>pricing_ilp.py</code> , <code>pricing_model_ortools.py</code>	02, 04	the repository starts from an optimization model rather than from a synthetic parity toy all downstream comparisons have an exact classical reference the mainline artifact path preserves optimization semantics
Exact baseline semantics	(12)	<code>classical_baselines.py</code> , <code>pricing_model_ortools.py</code>	02, 04	
Exact-reference encoding	Section 4.1	<code>ilp_to_xorsat_exact.py</code> , <code>verify_encoding.py</code>	04, 08a/08	
WHT surrogate encoding	(13)–(16)	<code>reduction.py</code>	04, 08a/08	exploratory spectral approximation can be diagnosed explicitly
Max-XORSAT DQI objective	(17)–(21)	<code>dqi_state.py</code> , <code>dqi_pipeline.py</code>	01, 03	the DQI artifact is mathematically explicit
Five-step DQI construction	(22)–(27)	<code>dqi_state.py</code> , <code>dicke_circuit.py</code>	01, 03	the quantum layer follows the source-paper construction
Exact/approximate decoding	(28)–(32)	<code>decoder_*.py</code> , <code>dqi_pipeline.py</code>	05, 09	decoder quality and postselection materially affect realized DQI behavior

Theory object	Key equations	Primary source files	Primary notebooks	Claim supported
Paper-derived alpha weights	(33)–(40)	weights_paper.py, weights.py, alpha_theory_validation.py	03, 10	the finite-size weight rule is implemented and validated
Resource model	(41)	resources.py, dicke_circuit.py, resources_compare.py	06, 12, 13	structure and decoder settings have interpretable resource consequences
Structural Qiskit subset	(42)	dqi_circuit_qiskit.py	07	cross-framework consistency is checked without overclaiming parity
Coherent tiny-instance checks	Section 11	coherent_reference.py	08b	appendix-only exactness boundary is documented
Multi-period extension	(43)	problem_generator_multiperiod.py, pricing_model_or_tools_multiperiod.py	10a, 11, 13	the modeling contract extends beyond the one-period toy family
Faithfulness dashboards and scaling	none beyond prior sections	metrics_decision.py, faithfulness.py, benchmark.py, matrix_runner.py, resources_compare.py	11, 12, 13	family-level benchmark conclusions remain semantically organized

No core src/ or notebook component in the current file map should remain theory-orphaned after this table.

18. Explicit caveats

1. The frozen pricing benchmark is inspired by the pricing problem of [2] but does not claim to reproduce its full industrial objective, demand model, or constraint set verbatim.
2. The DQI core, exact-decoding interpretation, and finite-size alpha rule are inherited from [1] and should only be called paper-faithful when their assumptions are respected.
3. The exact-reference encoding path is the mainline semantic backend; the Walsh path is exploratory.
4. The Qiskit notebook validates a structural subset unless decode/uncompute is actually included there.
5. Coherent validation and tiny-instance exactness checks are appendix-only and should not be misreported as the repository’s central application result.
6. Multi-period pricing is an extension layer, not the mainline DQI benchmark claim.

These caveats are part of the framework’s correctness, not disclaimers to be minimized.

19. Notation appendix

Symbol	Meaning
N_f	number of configurable features in the benchmark pricing model
n	number of decision bits
m	number of parity checks / max-XORSAT clauses
$x \in \{0, 1\}^n$	pricing configuration or assignment
$F(x)$	true benchmark pricing objective
$R(x)$	benchmark revenue term
$B \in GF(2)^{m \times n}$	parity matrix of the max-XORSAT artifact
$v \in GF(2)^m$	clause-sign / right-hand-side vector
$f(x)$	max-XORSAT objective used by DQI
P	degree- ℓ polynomial used by DQI amplitude shaping
ℓ	maximum error weight represented in the Dicke-state superposition
$ D_{m,k}\rangle$	Dicke state of Hamming weight k on m qubits
w_k	degree-allocation / alpha weight for weight class k
$C^\perp$	dual code defined by B^T
$d^\perp$	minimum distance of $C^\perp$
E_ℓ	set of error strings of Hamming weight at most ℓ
D_k	successfully decoded weight- k errors
F_k	unsuccessfully decoded weight- k errors
R	postselection normalization factor
G_{wt}	weighted Walsh-truncated surrogate
G_{unw}	sign-only max-XORSAT surrogate
d_{TV}	total variation distance

References

- [1] Stephen P. Jordan, Noah Shutty, Mary Wootters, Adam Zalcman, Alexander Schmidhuber, Robbie King, Sergei V. Isakov, Tanuj Khattar, and Ryan Babbush, *Optimization by Decoded Quantum Interferometry*, arXiv:2408.08292.
- [2] Francesc Sabater, Ouns El Harzli, Geert-Jan Besjes, Marvin Erdmann, Johannes Klepsch, Jonas Hiltrop, Jean-Francois Bobier, Yudong Cao, and Carlos A. Riofrio, *Towards solving industrial integer linear programs with Decoded Quantum Interferometry*, arXiv:2509.08328.
-

Final statement

The current repository is best understood as a mathematically stratified benchmark. The pricing layer defines the optimization problem. The exact-reference encoding layer preserves optimization semantics in the style advocated by [2]. The exploratory Walsh layer diagnoses what is lost when one infers a sparse parity surrogate directly from a truth table. The DQI core follows [1]. The decoder, alpha, resource, and faithfulness layers determine which statements are paper-faithful, which are benchmark-specific, and which belong only in appendices.

That stratification is the central theoretical contribution of the framework.